

PSU Dealing — คำอธิบายการทำงานทั้งระบบ

LINE OA "น้องติล": ผู้ช่วยหาคู่ + ที่ปรึกษาความรัก ด้วย Hybrid RAG (Dense + Graph) และ LLM แบบ Local + API

เอกสารฉบับนี้อธิบายทุกส่วนของระบบว่า ทำอะไร และ ทำเพื่ออะไร พร้อมตัวเลขที่วัดจริงในโปรเจกต์ เขียนจากโค้ดใน branch `faculty_score` ณ วันที่ 30 กันยายน 2569 (2026) ตัวเลขทุกตัวระบุที่มาไว้ ตัวเลขที่ยังไม่ได้วัดซ้ำหลังการเปลี่ยนแปลงล่าสุดจะเขียนกำกับไว้

สารบัญ

- ภาพรวมระบบและเป้าหมาย
- โครงสร้างโค้ดและหน้าที่ของแต่ละส่วน
- การรับ-ส่งข้อความผ่าน LINE
- การลงทะเบียนครั้งแรกและความยินยอม
- การแยกเจตนาของข้อความ (Intent)
- ความจำของบอท: บทสนทนาและความจำระยะยาว
- คลังความรู้ (Data & Knowledge Base)
- Dense RAG
- Knowledge Graph และ Neo4j
- การค้นความรู้แบบ Hybrid สำหรับตอบคำปรึกษา
- การสร้างคำตอบด้วย LLM และการกันตอบเกินคลังความรู้
- การเรียนรู้โปรไฟล์ผู้ใช้จากแชท
- การหาคู่ (Matching)
- หลังแนะนำคู่: ทำความรู้จัก / ขอผ่าน / เลิกคุย
- LLM แบบ Local และ API
- การประเมินผล
- ความเป็นส่วนตัวและความปลอดภัย
- ข้อจำกัดที่รู้อยู่และงานที่ควรทำต่อ
- ค่าส่งใช้งาน

1. ภาพรวมระบบและเป้าหมาย

1.1 ปัญหาที่ระบบแก้

นักศึกษา ม.อ. ต้องการ 2 อย่างที่มักแยกกันอยู่: (1) คนที่เข้ากันได้จริง ไม่ใช่แค่หน้าตาหรือคณะ และ (2) คำแนะนำเรื่องความสัมพันธ์ที่เชื่อถือได้ ไม่ใช่ความเห็นลอยๆ จากอินเทอร์เน็ต ระบบนี้รวมทั้งสองอย่างไว้ใน LINE OA ตัวเดียว

เพื่อให้ใช้ได้ ในแอปที่นักศึกษาเปิดอยู่แล้วทุกวัน ไม่ต้องติดตั้งแอปใหม่

1.2 ความสามารถหลัก

ปุ่ม / สิ่งพิมพ์	บอกทำอะไร	ทำอะไร
หาคู่	จัดอันดับคนที่เข้ากันด้วย Hybrid (Graph + Dense + คณะ + ค่านิยม) แล้วส่งการ์ดพร้อมเหตุผล	ให้ผู้ใช้เห็นว่า "ทำไม" ถึงแนะนำคนนี้ ไม่ใช่กล่องดำ
ถามบอด	ตอบจากคลังความรู้ 37 แหล่งพร้อมอ้างอิงชื่อเอกสาร ถามต่อเนื่องได้	คำแนะนำตรวจสอบได้ ไม่แต่งเอง
โปรไฟล์	แสดงทุกอย่างที่บอทจำเกี่ยวกับผู้ใช้	ความโปร่งใส ผู้ใช้รู้วาระบบเก็บอะไร
ล้างข้อมูล	ลบทุกอย่าง (หลังกดยืนยัน)	สิทธิ์ของเจ้าของข้อมูล
คุยเล่น	ตอบแบบเพื่อน และเก็บงานอดิเรก นิยาย สเปค คณะ ค่านิยม ลงโปรไฟล์ โดยอัตโนมัติ	ไม่ต้องกรอกแบบฟอร์มยาว โปรไฟล์ค่อยๆ ครบจากการคุย

1.3 ภาพรวมการไหลของข้อมูล

ข้อความ LINE

```
|
├─ แยกเจตนา (SBERT + BM25 + LLM เมื่อจำเป็น)
|
├─ ถามบอด → ขยายคำ / แปลงคำถาม → Router (Dense / Graph / Hybrid)
|           → ด้าน Reranker → LLM ตอบจาก context → ตรวจสอบหลักฐาน → ตอบ + อ้างอิง
|
├─ คุยเล่น → สกัดโปรไฟล์ (LLM Local) → อัปเดตโปรไฟล์ + Graph (Neo4j)
|
└─ หาคู่ → คัดคนที่เข้าเงื่อนไข → คะแนน Dense + Graph + คณะ + ค่านิยม
           → การ์ด + เหตุผล (LLM) → ทำความรู้จัก (ยินยอมทั้งสองฝ่าย)
```

ใช้ร่วมกัน: คลังความรู้ 1,035 chunk (ChromaDB) · Knowledge Graph ใน Neo4j (1,446 nodes / 8,252 relations)

ทำไมต้อง Hybrid: Dense (เวกเตอร์) เก่งเรื่อง "ความหมายคล้ายกัน" แต่ไม่ความสัมพันธ์เชิงโครงสร้าง เช่น "คน anxious กับคน avoidant มักมีปัญหากัน" หรือ "B ถูกรายงานว่โกหก 3 ครั้ง" ส่วน Graph รู้ความสัมพันธ์เหล่านี้แต่ค้นข้อความทั่วไปได้แย่ (Hit@5 ผังความรู้ของ Graph อย่างเดียวแค่ 0.122–0.294) การรวมกันจึงได้ทั้งสองจุดแข็ง

2. โครงสร้างโค้ดและหน้าที่ของแต่ละส่วน

โค้ดแยกโมดูลตามหน้าที่ ไม่รวมเป็นไฟล์ใหญ่ไฟล์เดียว เพื่อให้แก้ส่วนหนึ่งได้โดยไม่กระทบส่วนอื่น และให้แต่ละคนในทีมรับผิดชอบคนละส่วนได้

โฟลเดอร์ / ไฟล์	หน้าที่
-----------------	---------

app/server.py	FastAPI รับ webhook จาก LINE, โหลดโมเดลล่วงหน้าตอนเปิด server
app/handlers.py	ตัวกลาง: รับ event → แยกเจตนา → ส่งไปยัง flow ที่ถูกต้อง → ส่งคำตอบกลับ
app/intent.py, app/intent_model.py	แยกเจตนาด้วยความหมาย (SBERT + BM25 + LLM)
app/flows/	แต่ละความสามารถ: onboarding, chat, advice, match, intro, unmatch, account
app/conversation.py, app/storage.py	ความจำบทสนทนา / ความจำระยะยาว / ฐานข้อมูล SQLite
app/profile.py	โครงสร้างโปรไฟล์และการรวมข้อมูลใหม่เข้าโปรไฟล์
app/line_api.py, app/flex.py, app/rich_menu.py	คุยกับ LINE API, การ์ด Flex, Rich Menu
pipelines/ingest/	สร้างคลังความรู้: เก็บเอกสาร, OCR, ทำความสะอาด, ตัด chunk, ตัด concept
pipelines/dense/	embedding (bge-m3), ChromaDB, จัดรูปแบบ Dense
graph/	สร้าง Knowledge Graph, ค้นในกราฟ, คำนวณคู่จากกราฟ, เชื่อม Neo4j
pipelines/hybrid/	รวม Dense + Graph: router, ด้านความเกี่ยวข้อง, ขยายคำ, ค้นหา, จัดอันดับคู่
pipelines/llm/	prompt, เรียก LLM (Ollama / API), ตั้งค่าโมเดลทำงาน, benchmark
pipelines/model_eval/	เปรียบเทียบโมเดล SBERT / BERT / Local / API กลุ่มละ 5 ตัว + กราฟ
tests/, graph/tests/	unit test 96 ข้อ (แอป) + 56 ข้อ (กราฟ) + parity test ของ Neo4j

3. การรับ-ส่งข้อความผ่าน LINE

3.1 Webhook และลายเซ็น

LINE ส่งทุกเหตุการณ์ (ข้อความ, กดปุ่ม, เพิ่มเพื่อน, บล็อก) มาที่ `POST /callback` ระบบตรวจ ลายเซ็น **HMAC-SHA256** ด้วย channel secret ก่อนทุกครั้ง

ทำเพื่ออะไร: ป้องกันคนอื่นปลอมคำขอมาที่ server (เช่น ส่ง event "ลบข้อมูล" ของผู้ใช้คนอื่น) ถ้าลายเซ็นไม่ถูกต้อง ตอบ 401 ทันที

3.2 ตอบ 200 ทันที แล้วประมวลผลเบื้องหลัง

server ตอบ LINE ว่าได้รับแล้วทันที แล้วค่อยประมวลผลใน background task

ทำเพื่ออะไร: LINE จะส่งซ้ำถ้า webhook ตอบช้า การตอบก่อนป้องกันข้อความซ้ำ ส่วนคำตอบจริงส่งทีหลังด้วย reply token

3.3 Reply กับ Push

- **Reply** ใช้ reply token ที่มากับข้อความ ใช้ได้ครั้งเดียวและหมดอายุราว 1 นาที **ไม่นับโควตาข้อความ**
- **Push** ส่งหาใครก็ได้เมื่อไรก็ได้ แต่ **นับโควตา** ของ OA
- ระบบใช้ reply ถ้าประมวลผลเสร็จภายใน 50 วินาที ถ้าเกินหรือ reply ล้มเหลวจึงใช้ push

ทำเพื่ออะไร: ประหยัดโควตา และเป็นเหตุผลที่ระบบ **ไม่ส่งข้อความ "ขอผมคิดก่อนครับ..."** เพราะจะทำให้คำตอบจริงต้องเป็น push ทุกครั้ง

3.4 จุดแต่ง "กำลังพิมพ์" (Loading animation)

ก่อนเริ่มประมวลผลข้อความหรือการกดปุ่ม ระบบเรียก `/v2/bot/chat/loading/start` (60 วินาที) จุดแต่งหายเองทันทีที่คำตอบถึง

ทำเพื่ออะไร: คำถามบางข้อใช้ 5–20 วินาที ถ้าไม่มีสัญญาณอะไรผู้ใช้จะคิดว่าบอทค้าง จุดแต่ง **ไม่นับโควตา** และถ้าเรียกไม่สำเร็จก็ไม่ขวางการตอบ (แสดงเฉพาะแอปบนมือถือ ไม่แสดงบน LINE คอมพิวเตอร์ — ข้อจำกัดของ LINE)

3.5 ลำดับข้อความของผู้ใช้คนเดียวกัน

ข้อความของผู้ใช้คนเดียวกันถูกประมวลผลทีละข้อ (lock ต่อผู้ใช้)

ทำเพื่ออะไร: ถ้าผู้ใช้พิมพ์ 2 ข้อความติดกัน ข้อความที่สองต้องเห็นโปรไฟล์และประวัติที่ข้อความแรกบันทึกไว้แล้ว ไมอย่างนั้นข้อมูลจะเขียนทับกัน

3.6 Rich Menu

เมนู 4 ปุ่มด้านล่างแชท: **โปรไฟล์** (ส่งข้อความ "โปรไฟล์ของฉัน"), **ถามบอต**, **ล้างข้อมูล** (postback `action=delete_prompt`), **หาคู่** (ส่ง "หาคู่ให้หน่อย") ภาพขนาด 1520×1035 ตรวจสอบด้วย `app/check_line.py`

ทำเพื่ออะไร: ผู้ใช้ใหม่ไม่ต้องเดาว่าต้องพิมพ์อะไร และปุ่มลบใช้ postback ซึ่งผู้ใช้พิมพ์ปลอมเองไม่ได้

4. การลงทะเบียนครั้งแรกและความยินยอม

ลำดับ: เพิ่มเพื่อน → ข้อความต้อนรับ + **ขอความยินยอมใช้ข้อมูลเพื่อจับคู่** → เพศ → เพศที่สนใจ (ชาย / หญิง / ทุกเพศ) → อายุ (18–40) → คณะ → พร้อมใช้งาน

- ถ้า **ไม่ยินยอม** ยังปรึกษาเรื่องความรักได้ตามปกติ แต่จะไม่ถูกจับคู่ และไม่ถูกแนะนำให้ใครเห็น
- ผู้ใช้แต่ละคนได้เลขผู้ใช้ L0001, L0002, ... จากตัวนับใน DB ที่ **ไม่นำเลขกลับมาใช้ซ้ำ**

ทำเพื่ออะไร: ความยินยอมมาก่อนการเก็บข้อมูลเพื่อจับคู่ ส่วนเลขผู้ใช้ที่ไม่ซ้ำซ้ำกับจริง: เดิมใช้ "จำนวนผู้ใช้ + 1" เมื่อมีคนลบบัญชีแล้วคนใหม่สมัคร จะได้เลขซ้ำกับคนที่ยังอยู่ (ระบบล้มด้วย IntegrityError) และอาจได้เลขของคนที่ลบไป ซึ่งยังถูกอ้างถึงในข้อมูลของผู้ใช้อื่น

5. การแยกเจตนาของข้อความ (Intent)

5.1 ปัญหาของการดักคำตายตัว

เดิมใช้รายการคำ เช่น ข้อความมีคำว่า "ลบข้อมูลของฉัน" = สั่งลบ ปัญหาจริงจาก log: ผู้ใช้พิมพ์ "ลบโปรไฟล์" หรือ "ส่งลบโปรไฟล์" ซึ่งไม่อยู่ในรายการ ระบบจึงส่งไปตอบค่าปรึกษา แล้วตอบว่า "ไม่มีข้อมูลในคลัง" และหลังการ merge branch หนึ่ง ข้อความที่ไม่ใช่คำสั่ง **ทั้งหมด** ถูกส่งไปตอบค่าปรึกษา ทำให้แชทที่เรียนรู้โปรไฟล์ไม่ทำงานเลย

5.2 วิธีใหม่ 3 ชั้น (app/intent_model.py)

1. **SBERT (bge-m3):** แปลงข้อความเป็นเวกเตอร์ แล้วเทียบ cosine กับประโยคตัวอย่างของแต่ละเจตนา (8 เจตนา รว 6-12 ประโยคต่อเจตนา) คะแนนของเจตนา = ค่าเฉลี่ยของ 2 ตัวอย่างที่ใกล้ที่สุด
2. **BM25 บน character trigram:** คะแนนคำที่ใช้ร่วมกัน (ภาษาไทยไม่เว้นวรรคระหว่างคำจึงใช้ตัวอักษร 3 ตัวติดกัน) **ใช้แค่เสนอตัวเลือก ไม่ใช้ตัดสินใจ**
3. **LLM Local (qwen2.5 7B):** ใช้เฉพาะเมื่อ SBERT ก้ำกึ่ง (ความคล้ายสูงสุด < 0.55 หรือห่างอันดับสองน้อยกว่า 0.04) และเลือกได้เฉพาะจากตัวเลือกอันดับต้น

เจตนาทั้งหมด: find_match, show_profile, delete_me, unmatched, recall_memory, ask_advice, followup (→ ask_advice), chat

ทำไม BM25 ห้ามตัดสินใจ: วัตถุจริงแล้ว "ลบโปรไฟล์" SBERT ให้ delete_me 0.82 vs show_profile 0.75 (ถูก) แต่ trigram "โปรไฟล์" ตรงกับตัวอย่าง "ดูโปรไฟล์" ถ้าให้คะแนนรวมตัดสินใจจะกลายเป็นดูโปรไฟล์ ความต่างอยู่ที่คำกริยา "ลบ" ซึ่งเป็นเรื่องความหมาย

ทำไมใช้ LLM Local: ข้อความผู้ใช้ไม่ออกนอกเครื่อง และใช้เพียงราว 1 ใน 5 ข้อความ (ข้อความที่ชัดเจนตัดสินใจด้วย SBERT ใน ~14 ms)

5.3 ลบข้อมูลต้องกดยืนยันเสมอ

เมื่อเจตนาเป็น delete_me บอทส่งปุ่ม "ยืนยันลบ" / "ยกเลิก" ก่อนทุกครั้ง

ทำเพื่ออะไร: การตีความความหมายอาจพลาด และการลบย้อนกลับไม่ได้ การกดยืนยันทำให้ความผิดพลาดของตัวแยกเจตนาไม่กลายเป็นความเสียหาย

5.4 ผลวัด

วิธี	ชุดสด 26 ประโยค (ไม่ใช่ปรับค่า)	ชุดปรับค่า 49 ประโยค	เวลาต่อข้อความ
กฎคำตายตัว (เดิม)	50.0%	44.9%	0 ms
BM25 อย่างเดียว	96.2%	73.5%	~430 ms
SBERT อย่างเดียว	96.2%	91.8%	~14 ms
SBERT + BM25 + LLM (ใช้จริง)	100%	93.9%	~52 ms

ที่มา: `python -m app.intent_eval --fresh` ชุดปรับค่าใช้ปรับน้ำหนักจึงดูดีเกินจริง ตัวเลขหลักคือชุดสด ซึ่งมีเพียง 26 ข้อควรขยายก่อนรายงานเป็นทางการ

5.5 คำถามต่อเนื่อง

ข้อความอย่าง "ยกตัวอย่างหน่อย", "ข้อสองหมายถึงอะไร", "แล้วถ้าเขาไม่ตอบล่ะ" ต้องใช้บทสนทนาก่อนหน้า ถ้ามีประวัติบอกให้ LLM แปลงเป็นคำถามเดิมก่อนค้น ถ้าไม่มีประวัติและอ้างถึงสิ่งที่ไม่รู้ บอกถามกลับ ส่วนคำถามที่แค่ขึ้นต้นด้วย "เขา/เธอ" แต่ครบในตัว (เช่น "เขาไม่ตอบแชท ทำไงดี") ตอบเป็นคำถามใหม่ได้เลย

ยังเป็นกฎคำ: การตรวจคำถามต่อเนื่อง (`is_followup`) และการเปลี่ยนเรื่อง ยังใช้รายการคำ ควรย้ายไปใช้วิธีเดียวกับข้อ 5.2

6. ความจำของบอท

6.1 ความจำบทสนทนา (ระยะสั้น) — `storage.advice_history`

- ย้อนดูไม่เกิน 24 ข้อความล่าสุด เฉพาะคำปรึกษาและแชท
- หยุดย้อนเมื่อเจอจุดเริ่มหัวข้อ (คำถามปรึกษาที่ไม่ใช่คำถามต่อเนื่อง หรือผู้ใช้บอกเปลี่ยนเรื่อง)
- งบ 800 token: เก็บคำถามแรกของหัวข้อเสมอ + คู่มือสุด 3 คู่ ข้อความยาวตัดตรงกลาง เก็บต้นและท้าย (ท้ายมักมีคำแก้หรือคำปฏิเสธ)
- ลบเลขอ้างอิงและบรรทัดอ้างอิงออกจากคำตอบเก่า ช้อน LINE ID / อีเมล / เบอร์โทร
- ข้อความบอทที่ไม่ใช่คำตอบ ("ไม่มีข้อมูล", "หมายถึงเรื่องไหน") ถูกแทนด้วยหมายเหตุสั้นๆ

ทำไมต้องแทนข้อความ "ไม่มีข้อมูล": วัตถุประสงค์: ผู้ใช้ถาม "ยกตัวอย่างหน่อย" หลังบอทตอบไม่มีข้อมูล ขึ้นแปลงคำถามดึงรายการหัวข้อในข้อความ "ไม่มีข้อมูล" (ทฤษฎีความรัก รูปแบบความผูกพัน...) ไปใช้เป็นหัวข้อ แล้วตอบเรื่องที่ไม่เกี่ยวกับคำถามเดิมเลย

6.2 ความจำระยะยาว — `app/conversation.py`

- สรุปข้อความใหม่ครั้งละไม่เกิน 12 ข้อความ ด้วย **LLM Local เสมอ** แม้งานอื่นจะตั้งให้ใช้ API

- ข้อความที่มี LINE ID, เบอร์, อีเมล, ศาสนา, ฮาลาล, สีสัน ไม่ถูกเก็บเป็นความจำและไม่ถูกส่งให้โมเดล
- ดึงข้อเท็จจริงที่ตรงคำถามไม่เกิน 6 รายการ ถ้าผู้ใช้ถาม "จำได้ไหม / เคยบอก" จะค้นข้อความเก่าเพิ่ม
- ความจำถูกส่งให้ LLM พร้อมป้าย "ไม่ใช่หลักฐานความรู้" และถ้าโมเดลลื้ม ความจำเดิมไม่หาย

ทำเพื่ออะไร: ให้อบตเข้าใจบริบทของผู้ใช้ข้ามวันได้ โดยไม่เปลี่ยนความจำของผู้ใช้ให้กลายเป็น "ความรู้" ที่ใช้ตอบคำถามแทนเอกสาร

7. คลังความรู้ (Data & Knowledge Base)

7.1 แหล่งข้อมูลและเกณฑ์เลือก

37 แหล่ง: วิทยานิพนธ์ของจุฬาฯ 3 เล่ม, บทความวิชาการ, สไลด์, เอกสารสุขภาพทางเพศ, บทความคณะจิตวิทยา จุฬาฯ, โรงพยาบาลนวมารมย์, TDRI, Thai PBS, ตำรวจภูธรภาค 9, DLTV, งานวิจัย ThaiJO เป็นต้น

เกณฑ์: หน่วยงานรัฐ / มหาวิทยาลัย / วารสารวิชาการ / สถาบันวิจัย / โรงพยาบาล / สื่อสาธารณะ ไม่ใช้บล็อกไลฟ์สไตล์ ข้อความต้องดึงจากต้นฉบับตรงๆ ไม่สรุปหรือแต่ง ทุกแหล่งมีชื่ออ้างอิง

ตัดออก 8 แหล่งพร้อมเหตุผล: ธนาคารกรุงเทพ, BNH, ooca (บล็อกการดาวน์โหลดอัตโนมัติ — ไม่ลบการบล็อก), กรมสุขภาพจิต (หน้าเว็บโหลดเนื้อหาด้วยสคริปต์) และอีก 4 แหล่งที่ผู้ดูแลถอดออก

ทำเพื่ออะไร: คัดตอบอ้างอิงแหล่งให้ผู้ใช้เห็นทุกครั้ง คุณภาพของแหล่งจึงเป็นคุณภาพของคำตอบโดยตรง หมวดที่เพิ่มในรอบหลัง (ความปลอดภัยแอปหาคู่, red flag ช่วงแรก, การปฏิเสธ, คำเรียกขาน, ghosting, หึงหวง, LGBTQ+, love bombing) มาจาก log คำถามจริงที่บอทตอบไม่ได้

7.2 การเก็บเอกสารจากเว็บ — `pipelines/ingest/collect.py`

ดาวน์โหลด HTML → ดึงเฉพาะเนื้อหาบทความ → ตัดส่วนท้ายที่ไม่ใช่เนื้อหา (เครดิตผู้เขียน, แท็ก, รายการอ้างอิง, ลิงก์บทความอื่น) → สร้าง PDF ที่มีหัวอ้างอิง (ชื่อ / หน่วยงาน / URL / วันที่เข้าถึง) → ตรวจสอบว่าอ่านกลับได้ตรงต้นฉบับ

ปัญหาที่เจอและแก้: การสร้าง PDF ด้วยการจัดรูปอักษร (glyph shaping) ทำให้ไม่เอก ไม่โท ทัดขนาด หายจาก text layer ราว 7% ในทุกฟอนต์ไทยของเครื่อง จึงเปลี่ยนเป็นการวางตัวอักษรทีละตัว ผล: อ่านกลับได้ตรงต้นฉบับ 99.7–100% (หายเฉพาะอีโมจิ)

7.3 OCR สำหรับเอกสารสแกน

- ใช้ VLM (qwen2.5vl) อ่านภาพหน้าเอกสาร
- **ตัดขอบล่าง 6%** ก่อนอ่าน เพราะคลังวิทยานิพนธ์ประทับตรา "ดาวน์โหลดเมื่อ..." ทุกหน้า
- อ่าน 2 รอบ แล้วให้ **WangchanBERTa (masked LM ภาษาไทย)** เลือกคำที่เป็นไปได้กว่าในจุดที่ 2 รอบไม่ตรงกัน เลือกเฉพาะเมื่อมั่นใจ ที่เหลือเก็บไว้ให้คนตรวจ
- หน้าที่คนตรวจจากภาพแล้ว (101 หน้า) ใช้แทนผล OCR

ผล: CER 8.0% → 4.3%, WER 14.6% → 11.2% (วัดกับหน้าที่ถอดด้วยมือ `ocr_eval.py`)

ทำเพื่ออะไร: ตัวอักษรที่ OCR ผิดทำให้ค้นไม่เจอและทำให้คำตอบอ้างอิงข้อความผิด หน้าผลการวิจัย (บท 4–5) ซึ่งตอบคำถามเรื่องการเริ่มจับโดยตรงจึงถูกตรวจด้วยคนทุกหน้า

7.4 ทำความสะอาดข้อความ

แก้ฟอนต์ไทยเก่า (PUA) 20,165 จุด, แก้สระอาแตก, ลบบรรทัดขยะ (เลขหน้า หัวกระดาษ ตรา) 4,018 บรรทัด, ตัดเลขาอ้างอิงในเนื้อความ, ตัดอีเมลผู้เขียน

7.5 แบ่ง section และตัด chunk

- แบ่งตามบทของวิทยานิพนธ์ หรือตามหน้า รวม section ที่สั้นกว่า 80 คำกับ section ถัดไป
- chunk** ละ 300 คำ ช้อนกัน 15% (นับคำด้วย PyThaiNLP)
- ตัด chunk** ซ้ำ 18 อัน (ภาคผนวกที่พิมพ์แบบสอบถามชุดเดิมซ้ำ 5 เรื่องไขการทดลอง) เหลือ 1,035 chunk

	p50	p95	max
คำ (PyThaiNLP)	261	305	345
token (bge-m3)	323	441	962

ทำไม 300 คำ / 15%: ใหญ่พอให้ chunk หนึ่งมีทั้งประเด็นและคำอธิบาย (ย่อหน้าวิชาการไทยยาว) เล็กพอให้ใส่ 3–4 chunk ใน context ของ LLM · bge-m3 รับได้ 8,192 token ทุก chunk อยู่ในขีด · reranker อ่านได้ 512 token มี 1.6% ที่เกิน จึงอ่านเป็นช่วงละ 600 ตัวอักษรช้อนกัน · overlap ทำให้ประโยคที่คร่อมรอยตัดอยู่ครบใน chunk ใน chunk หนึ่ง · chunk ซ้ำแย่งที่ในผลค้น 5 อันดับแรก จึงต้องตัด

7.6 Metadata และ Concept

แต่ละ chunk มี: `chunk_id` คงที่, ที่มา (source / ชื่อ / ปี), ชนิดเอกสาร, ระดับความน่าเชื่อถือ, บท / หน้า, หมวด (ใช้ใน router), **concept** (taxonomy id ชุดเดียวกับโปรไฟล์ผู้ใช้), หัวข้อย่อย

ติด concept ด้วย lexicon + กฎบริบท:

- คำกำกวมต้องมีคำบริบทใกล้เคียง ภายใน ±60 ตัวอักษร (เช่น "มั่นคง" ต้องอยู่ใกล้ "ผูกพัน / secure")
- `not_after`: ไม่นับถ้าตามหลังคำปฏิเสธ ("ไม่มั่นคง" ไม่ใช่ secure)
- `not_before`: ไม่นับถ้าคำต่อท้ายทำให้เป็นคำอื่น ("ความมั่นคงปลอดภัยไซเบอร์", "รู้สึกผิดหวัง")
- ข้ามคำที่ถูกปฏิเสธ เช่น "ไม่รับผิดชอบ", "การไม่สนใจ", "ไม่พูดหลอกลวง"
- เอกสารการแพทย์และงานวิจัยภาษาศาสตร์ไม่ติด concept (คำว่า "สนธิ" ในงานภาษาศาสตร์ = ความคุ้นเคย ไม่ใช่ความใกล้ชิดแบบคู่รัก)

สุ่มตรวจ 50 คู่ (chunk, concept): ถูก 76% → แก้คำปฏิเสธแล้ว 80.9% ในชุดเดิม (ผู้ตรวจ: Claude — ควรให้คน

ตรวจยืนยันและสุ่มชุดใหม่) · ทดลองบังคับให้คำตรงรอยตัดคำแล้ว **ไม่ใช่** เพราะแก้ได้ 1 ข้อแต่ทำ tag ที่ถูกหายไป 6 ข้อ

ทำเพื่ออะไร: concept คือสะพานระหว่างคลังความรู้กับกราฟและโปรไฟล์ผู้ใช้ ถ้า tag ผิด กราฟจะดึงความรู้ที่ไม่เกี่ยวข้องมาตอบ รายละเอียดทั้งหมดอยู่ใน docs/data_card.md

8. Dense RAG

8.1 ฟังตอบเอกสาร

- ทุก chunk แปลงเป็นเวกเตอร์ด้วย **BAAI/bge-m3** (โมเดลหลายภาษาที่รองรับไทย) เก็บใน ChromaDB (cosine)
- ค้นด้วยทุกแบบของคำถาม (คำถามเดิม + คำพ้อง) แล้วรวมผล
- **จำกัด 3 chunk ต่อแหล่ง** — วิทยานิพนธ์เล่มใหญ่ (200+ chunk) เคยยึดผลค้นทั้งหมดจนแหล่งอื่นที่ตอบตรงหลุด

8.2 ฟังหาคู่

- แต่ละคนมีเวกเตอร์ "ตัวตน" (persona) และ "สเปคที่อยากได้" (preference)
- คะแนน = **ค่าเฉลี่ยฮาร์มอนิกสองทาง**: A อยากได้แบบ B และ B อยากได้แบบ A (ฮาร์มอนิกลงโทษคู่ที่ชอบอยู่ฝ่ายเดียว)
- หักคะแนนถ้าผู้สมัครคล้ายคนที่ผู้ใช้เคยปัดทิ้ง ($\lambda = 0.15$)

จุดอ่อนที่รู้: embedding แยกความหมายตรงข้ามได้แย่มาก เช่น "อยากมีลูก" กับ "ไม่อยากมีลูก" (cosine 0.70 vs 0.63) จึงต้องให้ LLM อ่านค่านิยมเป็นโครงสร้างแยก (ข้อ 12.3)

9. Knowledge Graph และ Neo4j

9.1 โครงสร้าง

ประเภทโหนด	ตัวอย่าง
User	ผู้ใช้จำลอง 300 คน + ผู้ใช้ LINE จริง
Concept	Hobby, Trait, CommStyle, Attachment, LoveLanguage, LoveComponent, RedFlag, ResearchFactor, BodyType, SkinTone, Hygiene
Source, BookChunk	เอกสาร และ chunk ของเอกสาร
Claim	ข้อความเชิงความสัมพันธ์ที่สกัดจากงานวิจัย (ใช้ได้เมื่อคนตรวจแล้ว)

ความสัมพันธ์	ความหมาย
HAS_TRAIT, LIKES, HAS_LOVE_LANGUAGE, HAS_LOVE_COMPONENT, HAS_FACTOR	คนมีลักษณะ / ชอบ
PREFERS, AVOIDS, SELF_DESCRIBED	สเปคที่อยากได้ / ไม่เอา / รูปลักษณะที่บอกเอง
REPORTED_AS	ถูกรายงานพฤติกรรม (นับเมื่อ ≥ 3 คน)
UNMATCHED, MATCHED, PASSED	ประวัติการจับคู่
COMPATIBLE_WITH, CONFLICTS_WITH, OPPOSITE_OF	กฎความเข้ากันระหว่าง concept (14 กฎ)
HAS_CHUNK, NEXT_CHUNK, ABOUT, SUPPORTED_BY, SUBJECT, OBJECT	เอกสาร $\rightarrow$ chunk $\rightarrow$ concept / หลักฐานของ Claim

ข้อควรทราบ: กฎความเข้ากัน 14 กฎยังเป็น "กฎของโปรเจกต์ที่ยังไม่ได้ยืนยันกับเอกสาร" (graph report ระบุเอง) ควรผูกแต่ละกฎกับงานวิจัยที่รองรับ

9.2 บทอ่านกราฟจาก Neo4j ทุกคำขอ — `graph/neo4j_view.py`

คลาส `Neo4jGraphView` มีหน้าตาเดียวกับกราฟในหน่วยความจำ (`GraphView`) ทุกการอ่านถูกแปลงเป็น Cypher ไปยัง Neo4j อัลกอริทึมค้นและให้คะแนนเดิมจึงใช้ได้ทันทีโดยไม่ต้องเขียนใหม่

- อ่านเฉพาะ **snapshot ที่ active** (ข้อมูลแต่ละชุดมี snapshot id จาก hash ของเนื้อหา)
- **ตั้งเป็นก่อน:** หาคู่ตั้งเส้นขาออกของผู้ใช้และผู้สมัครทั้งหมดใน query เดียว / ค้นความรู้ตั้ง property ของ chunk ที่ได้คะแนนใน query เดียว
- **ผู้ใช้ LINE** ถูกเขียนลง Neo4j (ป้าย `live=true`) เมื่อโปรไฟล์เปลี่ยน และ **ถูกลบออก (DETACH DELETE)** เมื่อผู้ใช้ลบข้อมูล
- **Neo4j ล่ม** $\rightarrow$ ใช้กราฟในหน่วยความจำแทนฮาร์ดไดรฟ์ + log เดือน · ตอนเปิด server เทียบ chunk ใน Neo4j กับ คลังความรู้ ถ้าไม่ตรง log ให้ import ใหม่

	ครั้งแรก (ยิงทีละโหนด)	หลังตั้งเป็นก่อน
ค้นความรู้ต่อคำถาม	~1,000 ms (1,034 queries)	~85 ms (11–15 queries)
หาคู่ 150 ผู้สมัคร	—	~300 ms

ความถูกต้อง: `graph/tests/test_neo4j_parity.py` ยืนยันว่าคะแนนคู่, เหตุผลบนการ์ด, กฎ และผลค้นความรู้จาก Neo4j ตรงกับกราฟในหน่วยความจำทุกคำ (4/4)

ทำไมผู้ใช้ LINE ติดป้าย live: ชั้น import ตรวจสอบว่าจำนวนโหนดตรงกับข้อมูลที่ import ถ้านับผู้ใช้ LINE รวมด้วย การ

import ชุดเดิมซ้ำจะล้ม · **ทำไมต้องลบใน Neo4j:** Neo4j เก็บถาวร ต่างจากกราฟในหน่วยความจำที่หายตอน restart ถ้าไม่ลบ ข้อมูลของคนที่ลบบัญชีไปแล้วจะค้างอยู่

10. การค้นคว้ามรู้แบบ Hybrid สำหรับตอบคำปรึกษา

10.1 ขยายคำ — `pipelines/hybrid/query_expand.py`

- **คำเรียกอีกฝ่าย:** ผู้ใช้พูด "คนคุย / คำ / คนที่ชอบ" เอกสารเขียน "แฟน / คู่รัก / อีกฝ่าย" → ค้นทั้งแบบเดิม และแบบแทนเป็นคำกลาง ("เพื่อน" แทนเมื่อไม่มีคำอื่น เพราะ "นัดเพื่อนกับแฟน" ต้องไม่กลายเป็นแฟนกับแฟน)
- **ภาษาพูด:** ทัก → เริ่มต้นความสัมพันธ์ / จีบ · ขอไลน์ → ขอเบอร์โทรศัพท์ · ขอยืมเงิน → หลอกให้โอนเงิน · หายเงียบ → ghosting ฯลฯ แทนเฉพาะคำเต็มหลังตัดคำ ("ทัก" ต้องไม่ไปแทนใน "ทักซะ")
- ไม่เกิน 5 แบบต่อคำถาม

วัดจริง: "ควรเริ่มทักยังไงดี" กับหน้าที่ตอบได้ ได้คะแนนความเกี่ยวข้อง 0.054 (ตกด่าน) → "ควรเริ่มต้นความสัมพันธ์ยังไงดี" 0.664 / "ควรเริ่มจีบยังไงดี" 0.884 · "คนที่คุยในแอปขอยืมเงิน" 0.01 → "ให้โอนเงิน" 0.37

10.2 Router — `pipelines/hybrid/router.py`

- ไม่พบ concept ในคำถาม → **dense** (คำถามข้อเท็จจริงทั่วไป)
- ถามความเข้าใจ ("เข้ากัน", "คบกับ", "ตรงข้าม") หรือพบ ≥ 2 concept → **graph**
- อื่นๆ → **hybrid**
- chunk หมวดสุขภาพทางเพศใช้ได้ เฉพาะเมื่อคำถามเกี่ยวกับสุขภาพ

ทำไมกันหมวดสุขภาพ: วัดจริง: "นัดเพื่อนกับแฟนวันเดียวกัน" เคยดึงเรื่องการสื่อสารกับคู่อุปสรรคมาอันดับ 5 · ผลหลังแก้: health_leak 0/30

10.3 ด้านความเกี่ยวข้อง — `pipelines/hybrid/gate.py`

ให้ **cross-encoder bge-reranker-v2-m3** อ่านคำถามคู่กับเอกสารทีละคู่ (อ่านจริงทั้งสองฝั่ง ต่างจาก SBERT ที่เทียบเวกเตอร์) แล้วเก็บเฉพาะเอกสารที่:

- ได้คะแนน ≥ 0.1 และ
- ได้คะแนน $\geq 30\%$ ของอันดับ 1 (ตัดขึ้นที่ห่างจากชั้นที่ตรงที่สุดมาก)

ใช้คะแนนสูงสุดของทุกแบบคำถาม · เอกสารยาวอ่านเป็นช่วงละ 600 ตัวอักษรซ้อน 150 · **กฎจากกราฟ** ถูกแปลงเป็นประโยคธรรมดา ก่อนให้คะแนน (ข้อความที่ส่งให้ LLM ไม่เปลี่ยน) · เรียงลำดับสุดท้ายตามคะแนนความเกี่ยวข้อง

ทำไมต้องมีด้านนี้: การค้นเวกเตอร์ "เจอเสมอ" แม้คำถามจะอยู่นอกคลัง (ค้นอย่างเดียว empty_on_unanswerable = 0/11) ด้านนี้คือขั้นแรกที่ทำให้บอตอบว่า "ไม่มีข้อมูล" ได้ · **ทำไมแปลงกฎเป็นประโยค:** reranker ไม่เข้าใจรูปแบบลูกศร "(A) -[CONFLICTS_WITH]-> (B)" วัดจริง "anxious กับ avoidant เข้ากันไหม" 0.00 → 0.96 · **ทำไมเรียงตามความเกี่ยวข้อง:** "ควรเริ่มทักยังไงดี" เคยได้ chunk ที่ตอบตรง (0.884) คู่กับบทความคนช้ำ (0.107)

10.4 รอบแปลงคำถาม (เมื่อรอบแรกไม่เจอ) — `pipelines/hybrid/search.py`

ถ้ารอบแรกได้เอกสารที่เกี่ยวข้องสูงสุด < 0.3 ให้ LLM เขียนคำถามใหม่ 3 บรรทัด:

1. คำถามเดิมที่ตัดรายละเอียดเฉพาะตัว (ชื่อ สถานที่ ตัวเลข จำนวนเงิน เหตุผลที่อีกฝ่ายอ้าง)
2. พฤติกรรมหลักของอีกฝ่าย + ถามว่าเป็นสัญญาณหรือความเสี่ยงอะไร
3. คำถามที่กว้างที่สุดของหัวข้อนั้น

LLM ได้รับ **รายชื่อหัวข้อในคลัง 36 หัวข้อ** ด้วย เพื่อเขียนด้วยคำแบบเอกสาร จากนั้นค้นใหม่ + ดึง chunk ก่อน/หลังของชิ้นที่ผ่านเข้ามาให้คะแนนด้วย ใช้ผลรอบ 2 เฉพาะเมื่อดีกว่ารอบแรก และ **ไม่ให้คะแนนคู่ที่รอบแรกให้ไปแล้วซ้ำ** (ผลเท่าเดิมทุกตัว เร็วขึ้นราว 25%)

ทำเพื่ออะไร: ผู้ใช้เล่าสถานการณ์เฉพาะตัว แต่เอกสารเขียนเป็นหลักการทั่วไป · วัตถุจริง: "คนคุยขอเงินไปจ่ายค่าเทอมควรให้ไหม" รอบแรก 0.00 → แปลงเป็น "การที่คนคุยขอเงินเป็นสัญญาณของ Romance Scam หรือไม่" ได้บทความตำรวจ 0.85 · "คำถามเรื่องสวนตัว เปลี่ยนเรื่องยัง'ใจดี'" เจอบทความ Healthy Boundary c04 (0.61) ที่เคยหลุดเพราะจำกัด 3 chunk ต่อแหล่ง จึงต้องดึง chunk ข้างเคียง · หัวข้อจาก LLM ใช้ค้นเท่านั้น คำตอบยังต้องผ่านด่านเดิมทุกด่าน

11. การสร้างคำตอบด้วย LLM และการกันตอบเกินคลังความรู้

11.1 กฎใน prompt (7 ข้อ)

1. ใช้ข้อมูลใน CONTEXT เท่านั้น ห้ามใช้ความรู้ของตัวเอง ห้ามแต่งเพิ่ม
2. ทุกประโยคที่ใช้ข้อมูลต้องลงท้ายด้วยเลขอ้างอิง [n]
3. ถ้าไม่มีข้อมูลที่ตอบได้ ให้ตอบ "ไม่มีข้อมูลเพียงพอ" เท่านั้น
4. ถ้า CONTEXT แค่ใกล้เคียงแต่ไม่ได้ตอบตรง ให้ตอบว่าไม่มีข้อมูล ห้ามหยิบเรื่องอื่นมาตอบแทน
5. สรุปด้วยคำพูดของตัวเองให้ตรงคำถาม ไม่คัดลอกข้อความยาว
6. คำเรียกอีกฝ่าย (คนคุย / คำ) = แฟน / คู่รัก / อีกฝ่าย ในเอกสาร
7. ห้ามเปลี่ยน "ตัวอย่าง" หรือเรื่องเล่าของคนอื่นเป็นคำแนะนำว่า "ควร..."

คำถามที่ส่งให้ LLM แนบคำฟ้องและหัวข้อที่เอกสารใช้ด้วย ("คำในเอกสารอาจใช้คำอื่นที่หมายถึงเรื่องเดียวกัน เช่น: ...") ส่วนประวัติแชทถูกส่งไปพร้อมกฎว่า "ใช้เพื่อเข้าใจสถานการณ์เท่านั้น ไม่ใช่หลักฐาน ใช้เลขอ้างอิงจาก CONTEXT รอบนี้เท่านั้น"

ทำไมต้องแนบคำฟ้อง: วัตถุจริง: แม้ context จะมีคำตอบ LLM ก็ตอบ "ไม่มีข้อมูล" สำหรับ "ควรเริ่มทักยัง'ใจดี" เพราะไม่รู้ว่ "ทัก" คือ "เริ่มต้นความสัมพันธ์" แนบแล้วตอบได้และผ่านการตรวจหลักฐาน

11.2 ตรวจหลักฐานหลังตอบ — `verify_answer`

ตัดคำตอบเป็นท่อน (≥ 40 ตัวอักษร) แล้วให้ cross-encoder เทียบแต่ละท่อนกับเอกสารที่ถูกอ้าง ท่อนที่ได้ < 0.1 หรือ

แค่ทวนคำถามถูกตัดทิ้ง ถ้าไม่เหลืออะไร → ตอบว่าไม่มีข้อมูล

ทำไมตรวจทีละท่อน ไม่ใช่ทีละย่อหน้า: LLM มักใส่อ้างอิงครั้งเดียวท้ายย่อหน้า ถ้าให้คะแนนทั้งย่อหน้า ท่อนที่แต่งเพิ่มจะได้คะแนนของท่อนจริงไปด้วย · วัตถุประสงค์: ประโยคแต่งเกิน "คุยแซ่บหน่อยๆ เพื่อให้ความสัมพันธ์ดีขึ้น" ได้ 0.008 ส่วนประโยคที่มีหลักฐานจริงต่ำสุด 0.568 เกณฑ์ 0.1 จึงแยกได้ชัด · **ทำไมตรวจคำถามคำถาม:** LLM เคยตอบแค่ "จิบรุ่มพีในคณะดีไหม [10]"

11.3 ทำความสะอาดก่อนส่ง

- ดัด markdown (**, #, `) เพราะ LINE ไม่แสดงผล
- ชื่อแหล่งที่ดูเหมือนโดเมนถูกทำให้ไม่เป็นลิงก์ (LINE แปลง "xxx.co" เป็นลิงก์อัตโนมัติ)
- บรรทัดท้าย "👉 อ้างอิง:" แสดงชื่อเอกสารที่ถูกอ้างอิงจริง ไม่เกิน 3 แหล่ง

11.4 สรุปการกันตอบเกิน RAG (3 ชั้น)

ชั้น	กันอะไร
ด่านก่อน LLM (reranker)	คำถามนอกคั้ง ไม่ส่งเอกสารที่ไม่เกี่ยวให้ LLM
Prompt 7 ข้อ	ห้ามใช้ความรู้ตัวเอง / ห้ามตอบเรื่องใกล้เคียงแทน / ห้ามเปลี่ยนตัวอย่างเป็นคำแนะนำ
ด่านหลัง LLM (verify)	ตัดท่อนที่ไม่มีหลักฐานในเอกสารจริง

ผล (rag_judge 52 ข้อ ล่าสุด): ปฏิเสธคำถามนอกคั้งถูก **11/11**, ตอบคำถามที่มีข้อมูล 36–37/41 (ต่างกันตามรอบ เพราะ API ไม่ deterministic), คำตอบมีหลักฐานและอ้างอิง 100%, **ข้อความที่แต่งเกินเอกสาร 0**

ข้อจำกัด: กรรมการที่ใช้ตรวจ (gemma3:12b) เคยปล่อยคำตอบที่แต่ง 2 ข้อผ่าน จึงต้องอ่านคำตอบเองประกอบ และควรมี gold set ที่คนตรวจ · กฎข้อ 4 แบบเข้มทำให้คำถามที่ต้องอนุมาน (เช่น "ชวนไปต่างจังหวัดหลังคุย 3 วัน ปกติไหม") ตอบว่าไม่มีข้อมูล ซึ่งเป็นการเลือกความปลอดภัยเหนือความครอบคลุม

12. การเรียนรู้โปรไฟล์ผู้ใช้จากแชท

12.1 สกัดโปรไฟล์ — `app/flows/chat.py`

ทุกข้อความแชท LLM Local (qwen2.5 7B, ผลลัพธ์ตาม JSON schema) สกัดงานอดิเรก นิสสัย สไตล์การสื่อสาร สิ่งที่ยกได้ สิ่งที่ไม่เอา เป็น taxonomy id แล้วรวมเข้าโปรไฟล์ ข้อความเรื่องคณะหรือนิยามจะไม่ถูกใช้เดานิสสัย (เช่น "วิศวะ" ไม่ได้แปลว่า "มีเหตุผล")

- ของเดิมเพิ่ม confidence และจำนวนหลักฐาน ข้อมูลไม่โตตามจำนวนแชท (งานอดิเรกไม่เกิน 8, นิสสัยไม่เกิน 6)
- confidence ลดครึ่งทุก 60 วันที่ไม่ถูกพูดถึง ต่ำกว่า 0.3 ถูกลบ

12.2 เปลี่ยนใจ

- คำว่า "เปลี่ยนใจ / เปลี่ยนเป็น / แก้เป็น / ไม่เอาแล้ว / ตอนนี้ชอบ / ...แทน" → กลุ่มที่เลือกได้ค่าเดียว (สีผิว, รูปร่าง) **ใช้ค่าใหม่แทนค่าเดิม** ส่วนนิสัยและงานอดิเรกไม่ลบของเดิม
- สิ่ง que เปลี่ยนเป็น "ไม่ชอบ" ถูกเอาออกจาก "อยากได้" และกลับกัน
- คณะ: เปลี่ยนใจ = แทนของเดิม, "ไม่จำกัดคณะ" = ล้างรายการ

ปัญหาจริง: "ผิวขาว" แล้ว "เปลี่ยนใจชอบผิวดำ" เคยได้โปรไฟล์ "ผิวขาว, ผิวเข้ม" ที่ขัดกันเอง · บั๊ทที่ทดสอบเจอ
ระหว่างแก้: "ผิวแทน" มีคำว่า "แทน" จึงไม่นับ "แทน" ที่อยู่หลัง "ผิว / สี" · **ข้อจำกัด:** คำว่าเปลี่ยนใจยังเป็นรายการค่า

12.3 คำนียม 11 มิติ

มิติ	ค่า
family	อยากมีลูก / ไม่อยากมีลูก
place	เมืองใหญ่ / บ้านเกิด ต่างจังหวัด
money	ประหยัด เก็บออม / ใช้จ่ายตามใจ
pets	รักสัตว์ / ไม่เลี้ยงสัตว์
goal	จริงจัง / ค่อยๆ ดูใจ
drinking, smoking	ดื่ม / ไม่ดื่ม · สูบ / ไม่สูบ
space	อยากใช้เวลาด้วยกันบ่อย / ต้องการเวลาส่วนตัว
social	ลงรูปคุในโซเชี่ยล / เก็บเป็นส่วนตัว
religion 🚫	พุทธ / อิสลาม / คริสต์ / ไม่นับถือ
diet 🚫	ฮาลาล / มังสวิรัต / กินได้ทุกอย่าง

LLM อ่านค่านียมจากข้อความเป็นโครงสร้าง ("ตัวเองเป็น" และ "อยากได้คนที่") **ศาสนาและอาหาร** เป็นข้อมูลอ่อนไหว (PDPA มาตรา 26): ยินยอมแล้ว = ใช้ / ยังไม่เคยถาม = พักไว้แล้วถามความยินยอม / ไม่ยินยอม = ทั้ง ประโยคที่มีข้อมูลอ่อนไหวไม่ถูกเก็บเป็นข้อความ ไม่เข้า Dense และไม่ส่งให้ LLM ไม่แสดงบนการ์ด

ทำไมให้ LLM อ่านเป็นโครงสร้าง: embedding แยกความหมายตรงข้ามไม่ออก (ข้อ 8.2) การอ่านเป็นค่าที่ชัดเจน ทำให้เทียบ "อยากมีลูก" กับ "ไม่อยากมีลูก" ได้ตรง

13. การหาคู่ (Matching)

13.1 คัดกรองก่อนจัดอันดับ (hard filter)

- ยินยอมให้จับคู่ทั้งสองฝ่าย
- เพศและเพศที่สนใจตรงกัน ทั้งสองทาง
- ไม่เคยปิด (pass) หรือเลิกคุย (unmatch) กันมาก่อน
- ผู้สมัครที่ถูกรายงาน red flag ≥ 3 คน ในเรื่องที่ใช้หลักเลียง ถูกตัดออก
- ไม่แนะนำคนที่เคยแนะนำไปแล้ว

13.2 คะแนน Graph — `graph/scorer.py`

feature	น้ำหนัก	ความหมาย
prefers	0.35	สิ่งที่แต่ละฝ่ายอยากได้ ตรงกับนิสัยของอีกฝ่าย (เฉลี่ยสองทาง)
theory	0.25	กฎความเข้ากันระหว่างลักษณะของสองคน (บวก = เข้ากัน, ลบ = ชัดกัน)
hobby	0.20	งานอดิเรกร่วม (Jaccard)
love_language	0.10	ภาษารักร่วม
lifestyle	0.10	การนอน / วันหยุดตรงกัน (ไม่รู้ = ไม่ได้คะแนน)

และคะแนนรูปลักษณ์ (สเปคที่อยากได้ / ไม่เอา เทียบกับสิ่งที่อีกฝ่ายบอกเอง ไม่เดาจากรูป)

13.3 คะแนน Hybrid ที่แอปใช้จริง

`HybridConfig(fusion="weighted", alpha=0.3, w_values_struct=0.3, w_faculty=0.3)`

- รวม Dense (30%) กับ Graph (70%) แบบถ่วงน้ำหนักหลังปรับสเกล
- บวกโบนัสค่านิยมแบบโครงสร้าง (0.3) และคนที่ผู้ใช้อยากได้ตรงกับคณะของอีกฝ่าย (0.3) — **คณะเป็นคะแนนบวก ไม่ใช่เงื่อนไขตัดทิ้ง**

วิธี (288 โปรไฟล์ที่มีผล)	P@5	MRR
Dense	0.113	0.261
Graph	0.261	0.547
Hybrid RRF พื้นฐาน	0.201	0.425
Hybrid weighted (Dense 0.3)	0.247	0.530

Hybrid ที่แอปใช้ (+ คำนิยม + คณะ)	0.320	0.638

ที่มา: docs/evaluation-2026-09-28.md · ข้อมูลผู้ใช้เป็นข้อมูลสังเคราะห์ จึงวัดกลไก ไม่ใช่ความแม่นยำในโลกจริง · Hybrid RRF พื้นฐานต่อยกกว่า Graph เดี่ยว ความต่างของ config ที่ใช้จริงมาจากสัญญาณคำนิยมและคณะ ไม่ใช่ fusion อย่างเดียว

13.4 การ์ดแนะนำ

การ์ด Flex แสดงชื่อ อายุ คณะ นิสัย งานอดิเรก เปอร์เซนต์ความเข้ากัน และ **เหตุผลไม่เกิน 3 ข้อ** จากกราฟ ("ทั้งคู่ชอบ...", "คุณอยากได้คนที่... — อีกฝ่ายมีนิสัยนี้", "เรียนคณะ... ตรงกับที่คุณอยากได้", "คำนิยมตรงกัน...") พร้อมเคล็ดลับเริ่มคุยที่ LLM (psu-gemma) เขียนจากเหตุผลเหล่านั้น

ทำเพื่ออะไร: เหตุผลมาจากเส้นทางในกราฟจริง ตรวจสอบได้ ไม่ใช่ LLM แต่งขึ้น · ไม่เปิดเผยรูปลักษณ์ สีผิว ศาสนา อาหาร บนการ์ด

14. หลังแนะนำคู่

14.1 ทำความรู้จัก (mutual consent) — app/flows/intro.py

1. A กด "สนใจทำความรู้จัก" บนการ์ดของ B
2. ระบบส่งการ์ดของ A ให้ B ประเมิน **ยังไม่มีช่องทางติดต่อของใคร**
3. B ยินยอม → ทั้งคู่ได้ LINE ID ของกันและกัน / B ไม่สะดวก → A ได้ข้อความสุภาพ ไม่ได้ข้อมูลของ B และระบบไม่แนะนำคู่นี้อีก

ทำเพื่ออะไร: ความปลอดภัยของทั้งสองฝ่าย ไม่มีใครได้ช่องทางติดต่อของคนที่ไม่ยินยอม · LINE API ไม่เปิดให้บอกรู้เห็น LINE ID ของผู้ใช้ จึงให้ผู้ใช้ส่ง ID เอง เก็บแยก และส่งต่อเฉพาะเมื่อยินยอมทั้งคู่

14.2 ขอฟ่าน

บันทึก event "pass" → ไม่แนะนำคู่นี้อีก และคนที่คล้ายคู่นี้จะถูกหักคะแนน Dense เล็กน้อย

14.3 เลิกคุย — app/flows/unmatch.py

1. ถามเหตุผลสั้นๆ
2. LLM แยกเหตุผลเป็น: พฤติกรรมของอีกฝ่าย / สเปคของผู้พูด (อยากได้-ไม่เอา) / รูปลักษณ์
3. **นโยบาย:** เฉพาะพฤติกรรม (red flag) เท่านั้นที่นับเป็นการรายงานอีกฝ่าย และมีผลเมื่อรายงาน ≥ 3 คน ส่วนเรื่องรูปลักษณ์ / สเปคใช้ปรับสเปคของผู้พูดเท่านั้น ไม่ลงโทษอีกฝ่าย
4. ครั้งต่อไปลดน้ำหนักและคัดกรองคนที่มีพฤติกรรมแบบนี้

ทำไมต้อง ≥ 3 คนและไม่นับรูปลักษณ์: กันการกลั่นแกล้งด้วยการรายงานคนเดียว และไม่ลงโทษใครเพราะหน้าตา

หรือสืฟว ซึ่งเป็นความชอบส่วนตัวของผู้พูด ไม่ใช่ข้อบกพร่องของอีกฝ่าย

15. LLM แบบ Local และ API

15.1 เลือกโมเดลต่องาน

งาน	โมเดลที่แนะนำ (มีผลวัด)	เหตุผล
ตอบคำปรึกษา	API psu-gemma (สำรอง Typhoon2 8B)	Typhoon เคยแต่งเลขอ้างอิงเกินและลอก chunk ดิบ · psu-gemma 0 เครดิต
อธิบายคู่ (การ์ด)	API psu-gemma	2.2 s vs Local 12.5 s · คะแนน 4.73 vs 4.53
แปลงคำถามค้นหา	API psu-gemma	qwen2.5:3b เคยลอกตัวอย่างใน prompt มาเป็นคำถาม
สกัดโปรไฟล์ / ความจำ	Local qwen2.5 7B	ข้อมูลส่วนตัวไม่ออกนอกเครื่อง
แยกเจตนา (ก้ากึ่ง)	Local qwen2.5 7B	ชุดสดถูก 100% · ใช้ ~1 ใน 5 ข้อความ
คุยเล่น	Local Typhoon2 8B (สำรอง 3B)	ภาษาไทยเป็นธรรมชาติ

สิ่งที่ต้องตรวจสอบก่อนนำเสนอ: ไฟล์ `.env` ณ วันที่เขียนตั้งให้ตอบคำปรึกษาใช้ **gpt-5.6-luna** (เครดิต ×2 ราว 1,900–4,400 เครดิตต่อคำถาม) และให้แปลงคำถามใช้ **qwen2.5:3b** ซึ่งต่างจากตารางข้างบน ควรเลือกให้ตรงกันและมีผลวัดรองรับ (`pipelines/model_eval`)

15.2 กลไกของ API — `pipelines/llm/api_client.py`

- ตั้งใน `.env` ด้วยค่านำหน้า `api:` เช่น `LLM_RAG_ANSWER=api:PSU-LLM/psu-gemma`
- ถ้า API ล่ม สลับไปใช้โมเดล Local เดิมของงานนั้นอัตโนมัติ บอกยังตอบได้
- หมุนเวียน **API key 3 ตัว**: key ที่ใช้ไม่ได้ (401/403) พัก 1 ชั่วโมง, ถูกจำกัด (429) พักตาม Retry-After, 5xx / ต่อไม่ได้ พัก 15 วินาที, ไม่มีโมเดล (404) หยุดทันที
- บันทึก token, เครดิต (`token × ตัวคูณของโมเดล`) และค่าใช้จ่ายลง log ทุกคำถาม ไม่มี key ใน log

15.3 เวลาตอบ (วัดจริง 1 คำถามที่ต้องแปลงคำถาม)

ขั้น	เวลา	สัดส่วน
แยกเจตนา	0.26 s	2%
ค้นรอบ 1 + reranker	1.89 s	17%
แปลงคำถาม + ค้นรอบ 2 + reranker	4.39 s	41%

LLM ตอบ (Luna)	3.80 s	35%
ตรวจหลักฐาน	0.44 s	4%
รวม	10.8 s	

ทางเร่งที่เหลือ (ยังไม่ได้ทำ): แคมข้อมูลที่ไม่เปลี่ยนของ Neo4j (~1 s), reranker แบบ fp16 (~0.5–1 s), ตอบด้วย psu-gemma (~1.5 s + 0 เครดิต), แคมคำตอบของคำถามที่คล้ายเดิม (~0.1 s สำหรับคำถามซ้ำ) · LINE รับคำตอบได้ครั้งเดียวต่อข้อความ จึงส่งคำตอบที่ละส่วนไม่ได้

16. การประเมินผล

วัดอะไร	เครื่องมือ	ผลล่าสุด
คุณภาพคำตอบและการหลอน	<code>pipelines.llm.bench.rag_judge</code> (52 คำถาม)	ปฏิเสธถูก 11/11, ตอบได้ 36–37/41, แต่งเกิน 0
การค้นความรู้	<code>pipelines.hybrid.knowledge_eval</code>	Hit@5: Dense 0.854, Graph 0.122, Hybrid RRF 0.902, Routed 0.902 (ก่อนตัด chunk ซ้ำ)
การจับคู่	<code>pipelines.hybrid.evaluate</code>	P@5: Dense 0.113, Graph 0.261, Hybrid ที่ใช้จริง 0.320
แยกเจตนา	<code>app.intent_eval --fresh</code>	กฎคำ 50% → 100%
OCR	<code>pipelines.ingest.ocr_eval</code>	CER 8.0% → 4.3%
concept tag	<code>pipelines.ingest.tag_audit</code>	76% → 80.9% (ชุดเดิม)
Neo4j = กราฟในหน่วยความจำ	<code>graph.tests.test_neo4j_parity</code>	4/4
เปรียบเทียบโมเดล 4 กลุ่ม × 5	<code>pipelines.model_eval.run</code>	ยังไม่ได้รัน
unit test	<code>unittest discover</code>	แอป 96 ข้อ, กราฟ 56 ข้อ ผ่านทั้งหมด

ข้อควรระวังในการอ่านผล: ความเกี่ยวข้องวัดด้วย "คำคาดหวังครึ่งหนึ่งอยู่ในเอกสาร" ซึ่งเป็นตัวแทน ไม่ใช่คนตรวจ · API ให้ผลไม่เหมือนกันทุกครั้ง ควรรันซ้ำ 3 รอบแล้วรายงานค่าเฉลี่ย · ข้อมูลผู้ใช้เป็นข้อมูลสังเคราะห์

17. ความเป็นส่วนตัวและความปลอดภัย

- **ความยินยอม:** ก่อนจับคู่ และแยกอีกครั้งสำหรับศาสนา / อาหาร / สัตว์
- **ไม่ส่งข้อมูลออนไลน์ให้ API:** ศาสนา อาหาร แชนดิบ LINE ID เบอร์ อีเมล · ความจำสรุปด้วย LLM Local
- **สิทธิ์ลบข้อมูล:** ลบโปรไฟล์ ความจำ ประวัติแชท event คำแนะนำ ช่องทางติดต่อ และโน้ตใน Neo4j ต้องกดยืนยันก่อน
- **ความโปร่งใส:** ผู้ใช้ดูได้ว่าระบบทำอะไร อีกฝ่ายไม่เห็นสิ่งที่ผู้ใช้คุยกับบอท
- **ช่องทางติดต่อ:** ส่งต่อเฉพาะเมื่อยินยอมทั้งสองฝ่าย
- **ความลับของระบบ:** `.env` (API key, LINE token), `graph/.env` และฐานข้อมูลแท้จริง (`*.db`) อยู่ใน `.gitignore`
- **ลิขสิทธิ์:** เอกสารส่วนใหญ่มีลิขสิทธิ์ ใช้เพื่อการศึกษา repo ต้องเป็น private และคำตอบอ้างอิงแหล่งทุกครั้ง
- **คำตอบเรื่องสุขภาพ:** ตอบจากเอกสารการแพทย์เท่านั้น และไม่ถูกดึงไปตอบคำถามเรื่องความสัมพันธ์ทั่วไป

18. ข้อจำกัดที่รู้ๆและงานที่ควรทำต่อ

1. **คำถามที่ต้องอนุมาน** ตอบว่าไม่มีข้อมูล (กฎข้อ 4 แบบเข้ม) — ทางแก้: เพิ่มเอกสาร หรือฟอนกฎโดยต้องบอกว่า "เอกสารไม่ได้พูดถึง"
2. **API ไม่ deterministic** ผลต่อข้อสลับได้ระหว่างรอบ — ทางแก้: จำค่าที่แปลงแล้วของคำถามเดิม และรายงานผลเฉลี่ยหลายรอบ
3. **ยังใช้กฎคำ:** คำถามต่อเนื่อง, การเปลี่ยนเรื่อง, การเปลี่ยนใจ, คำสัญญาสุขภาพใน router
4. **กฎความเข้ากัน 14 กฎ** ยังไม่ผูกกับหลักฐานงานวิจัย
5. **concept tag** ยังผิดความหมายบางกรณี และ tag จากบรรณานุกรม — ต้องให้คนตรวจ
6. **คำตอบเรื่องสุขภาพเร่งด่วน** (เช่น ถูยงแตก) ควรมีกฎให้แนะนำพบแพทย์ โดยอ้างอิงจากเอกสาร
7. **ยังไม่มี gold set** ที่คนตรวจ, ชุด red-team (prompt injection / ขอข้อมูลผู้ใช้อื่น), ผลเปรียบเทียบโมเดล 4 กลุ่ม, latency p50/p95 ระดับระบบ
8. **ข้อมูลผู้ใช้เป็นข้อมูลสังเคราะห์** ผลจับคู่วัดกลไก ไม่ใช่ความพึงพอใจของผู้ใช้จริง

19. คำสั่งใช้งาน

```
# เปิดระบบ
docker compose -f graph/compose.yaml up -d          # Neo4j (ต้องเปิดก่อน)
ollama serve                                          # โมเดล Local
.venv/bin/uvicorn app.server:api --host 0.0.0.0 --port 8000

# สร้างคลังความรู้ใหม่ (หลังเพิ่ม / แก้เอกสาร)
.venv/bin/python -m pipelines.ingest.run             # สกัด → chunk → concept
.venv/bin/python -m pipelines.dense.run build        # ChromaDB
.venv/bin/python -m graph.build                     # ตรวจสอบกราฟ
.venv/bin/python -m graph.import_neo4j              # ส่งกราฟเข้า Neo4j (บอทอ่านจากที่นี้)
.venv/bin/python -m pipelines.ingest.data_card      # สร้าง docs/data_card.md ใหม่
```

ประเมินผล

```
.venv/bin/python -m pipelines.llm.bench.rag_judge      # คุณภาพคำตอบ / การหลอน
.venv/bin/python -m app.intent_eval --fresh           # แยกเจตนา
.venv/bin/python -m pipelines.model_eval.run --groups sbert,bert,local
.venv/bin/python -m unittest discover -s tests        # unit test
.venv/bin/python -m unittest graph.tests.test_neo4j_parity
```

เอกสารที่เกี่ยวข้อง: docs/data_card.md (คลังความรู้), docs/checklist.md (สถานะตามเกณฑ์คะแนน),
docs/evaluation-2026-09-28.md (ผลประเมิน), graph/README.md (กราฟและ Neo4j),
docs/PSU_Dealing_Slides.pptx (สไลด์)